



OSHI BEAUTY: CURATED LUXURY SKINCARE AND MAKEUP

PROJECT REPORT

SUBMITTED BY

KODAGODA O.D. - EG/2023/5683

SUBMITTED ON

27TH OF JULY 2026

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE MODULE
EC3404 – GUI PROGRAMMING

SUBMITTED TO

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING
FACULTY OF ENGINEERING, UNIVERSITY OF RUHUNA
SRI LANKA

CONTENTS

CONTENTS	2
1. PROJECT OVERVIEW	4
1.1 Project Type	4
1.2 Domain.....	4
1.3 Objective.....	4
1.4 Target Audience.....	4
2. FEATURES IMPLEMENTED	5
2.1 Core Features	5
2.2 Pages Implemented.....	5
2.3 Shopping Cart Features.....	6
2.3 Bundle Offers	6
3. COMPONENT ARCHITECTURE.....	8
3.1 Component Hierarchy.....	8
3.2 Data Flow	9
3.3 Key Components	9
4. CONCEPTUAL DESIGN	10
4.1 DummyJSON Endpoints Used.....	10
4.2 Product Categories.....	10
4.3 TypeScript Product Interface.....	10
4.4 API Response Handling.....	11
5. DESIGN CHOICES	12
5.1 Color Scheme	12
5.2 Typography	12
5.3 Dark Mode Implementation	12
5.4 Responsive Design	13
5.5 User Experience Features	13
6. STATE MANAGEMENT	14
6.1 Cart Store Structure	14
6.2 Store State.....	14
6.3 Getters.....	15
6.4 Actions	15
6.5 Why Pinia Was Selected	15
7. CHALLENGES AND SOLUTIONS.....	16

8. TECHNOLOGY STACK.....	17
8.1 Frontend Technologies.....	17
8.2 State Management and Routing	17
8.3 API and Data Technologies.....	17
8.4 Development Tools.....	18
9. TESTING AND VALIDATION.....	19
9.1 Manual Testing Results.....	19
10. CONCLUSION.....	20
10.1 Project Summary	20
10.2 Learning Outcomes.....	20
11. FUTURE ENHANCEMENTS	21
11.1 User Authentication.....	21
11.2 Order History	21
11.3 Wishlist Functionality	21
11.4 Product Reviews and Ratings	21
11.5 Online Payment Gateway Integration.....	21
11.6 Admin Dashboard	21
11.7 Email Notifications	21
11.8 Product Recommendations	21
12. REFERENCES	22
12.1 API Documentation	22
12.2 Frameworks and Libraries	22
12.3 Additional Resources	22
13. ACKNOWLEDGEMENTS.....	23

1. PROJECT OVERVIEW

1.1 Project Type

Single Page Application (SPA)

1.2 Domain

Beauty and Cosmetics E-commerce

1.3 Objective

The objective of this project is to develop a fully functional, responsive, and data-driven Single Page Application (SPA) using Vue 3, TypeScript, and Tailwind CSS. The application consumes product data from the DummyJSON public API and implements essential e-commerce functionalities such as product browsing, product search, shopping cart management, and a simulated checkout process. The project demonstrates modern frontend development practices while providing users with an engaging online shopping experience.

1.4 Target Audience

The application is designed for beauty enthusiasts and online shoppers who are looking for a premium and user-friendly online platform to browse and purchase skincare, makeup, and fragrance products.

2. FEATURES IMPLEMENTED

2.1 Core Features

Feature	Description
Vue 3 Composition API	Built using the modern Composition API with <script setup> syntax for better code organization and reusability.
TypeScript	Ensures type safety and improves code maintainability through strongly typed interfaces.
Tailwind CSS	Provides utility-first responsive styling with built-in dark mode support.
DummyJSON API	Fetches real-time product data from the public DummyJSON REST API.
Vue Router	Enables client-side routing with smooth page navigation and scroll behavior.
Pinia	Manages global application state, including shopping cart functionality with localStorage persistence.
Dark Mode	Allows users to switch between light and dark themes while saving their preference locally.

2.2 Pages Implemented

Page	Route	Description
Homepage	/	Displays the hero section, promotional banner, featured products, and newsletter subscription.
Skincare	/skincare	Displays skincare products with sorting options such as price, name, and rating.
Makeup	/makeup	Displays makeup products with sorting functionality.
Fragrance	/fragrance	Displays fragrance products with sorting functionality.
Gifts	/gifts	Displays a "Coming Soon" page for future gift collections.
Offers	/offers	Displays bundle offers with discounts ranging from 15% to 32%.

About Us	/about	Presents the company story, mission, and brand values.
Cart	/cart	Displays selected products with quantity controls, bundle information, and total calculation.
Checkout	/checkout	Collects customer shipping information and payment preferences.
Payment	/payment	Simulates secure card payment with form validation.
Order Success	/order-success	Displays order confirmation, order number, and estimated delivery information.
Login	/login	Allows existing users to sign in.
Sign Up	/signup	Allows new users to create an account.
Search	/search	Provides real-time product search with suggestions.
Product Detail	/product/:id	Displays complete product information with quantity selection and add-to-cart functionality.

2.3 Shopping Cart Features

The shopping cart module includes several important functionalities that improve the shopping experience.

- Users can add products to the cart while selecting a quantity between 1 and 99.
- Individual products or complete product bundles can be removed from the cart.
- Product quantities can be increased or decreased using increment and decrement buttons.
- Bundle discounts are automatically calculated and applied.
- Cart information is stored using Local Storage to preserve data across browser sessions.
- The navigation bar displays a dynamic cart badge showing the total number of items.
- Bundle products are displayed separately with their original price, discount percentage, and final discounted price.

2.3 Bundle Offers

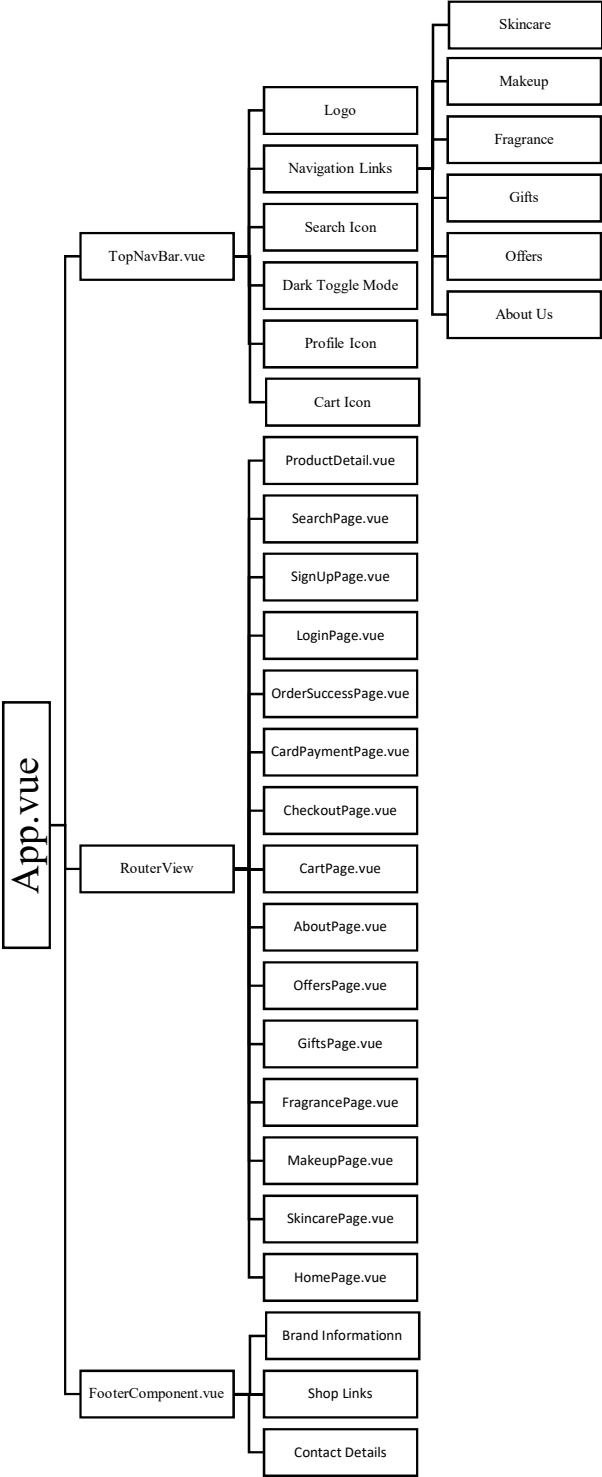
Bundle	Products Included	Discount	Description
Skincare Bundle	2 Products	15% OFF	Purchase two skincare products and receive a 15% discount.
Makeup Bundle	3 Products	25% OFF	Purchase three makeup products and receive a 25% discount.

Fragrance Bundle	2 Products	20% OFF	Purchase two fragrance products and receive a 20% discount.
Complete Beauty Set	Skincare + Makeup + Fragrance	32% OFF	Purchase one product from each category as a complete beauty package and receive a 32% discount.

3. COMPONENT ARCHITECTURE

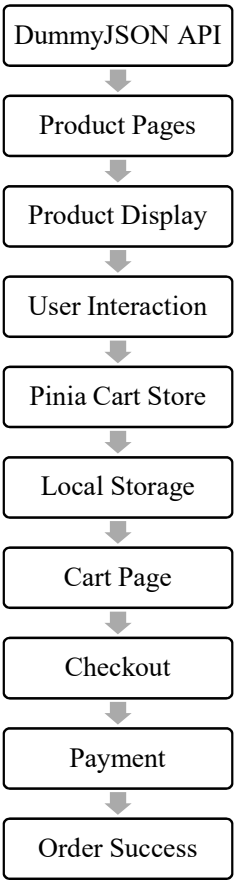
The application follows a modular component-based architecture using Vue 3. Each component is responsible for a specific functionality, making the application easier to maintain, debug, and extend. Vue Router dynamically renders different pages while shared components such as the navigation bar and footer remain consistent throughout the application.

3.1 Component Hierarchy



3.2 Data Flow

The application follows a simple and efficient data flow to retrieve and display product information.



3.3 Key Components

Component	Purpose	Key Features
TopNavBar	Main navigation component	Navigation links, search icon, dark mode toggle, login button, shopping cart badge
FooterComponent	Website footer	Company information, useful links, contact information
Product Pages	Display product collections	Responsive product grid, sorting options, Add to Cart button
ProductDetail	Individual product page	Product images, detailed description, price, quantity selector
CartPage	Shopping cart	Quantity management, item removal, subtotal and total calculations
OffersPage	Bundle offers	Displays bundle promotions with discount calculations

4. CONCEPTUAL DESIGN

The project integrates with the DummyJSON REST API to retrieve real-time product information. This eliminates the need for manually maintaining product data and demonstrates how external APIs can be consumed within a Vue.js application. represents the real-world operations of the food donation redistribution system.

4.1 DummyJSON Endpoints Used

Endpoint	Method	Purpose
/products/category/skin-care	GET	Retrieve skincare products
/products/category/beauty	GET	Retrieve makeup products
/products/category/fragrances	GET	Retrieve fragrance products
/products/{id}	GET	Retrieve detailed information about a selected product
/products/search?q={query}	GET	Search products based on user input

4.2 Product Categories

Category	DummyJSON Category	Application Page	Approximate Products
Skincare	skin-care	Skincare Page	30+
Makeup	beauty	Makeup Page	30+
Fragrance	fragrances	Fragrance Page	10+

4.3 TypeScript Product Interface

The following interface was used to ensure type safety while handling product information retrieved from the API.

```
interface Product {  
  id: number;  
  title: string;  
  description: string;  
  category: string;  
  price: number;  
  discountPercentage: number;  
  rating: number;  
  stock: number;  
  brand: string;
```

```
    thumbnail: string;  
    images: string[];  
}
```

4.4 API Response Handling

The application includes proper handling for different API response scenarios to improve user experience.

- **Successful Response:** Products are displayed in a responsive grid with images, prices, ratings, and discount information.
- **Loading State:** A loading spinner is displayed while data is being retrieved.
- **Error State:** User-friendly error messages are shown with an option to retry the request.
- **Empty Results:** If no matching products are found, an informative "No Products Found" message is displayed.

5. DESIGN CHOICES

The user interface was designed to provide an elegant and premium shopping experience while maintaining simplicity and responsiveness.

5.1 Color Scheme

Color	Hex Code	Purpose
Primary	#0F426F	Main headings, buttons, branding
Secondary	#52606E	Subheadings and secondary text
Light Background	#F8F9FA	Main application background
Dark Background	#1A1A2E	Background in dark mode
Light Text	#191C1D	Body text in light mode
Dark Text	#E8E8E8	Body text in dark mode
Accent	#2E5A88	Hover effects and gradients

5.2 Typography

Text Type	Font Family	Usage
Headings	Noto Serif	Titles and section headings
Body Text	Plus Jakarta Sans	Paragraphs and product descriptions
Labels	Plus Jakarta Sans	Buttons, navigation, badges, and form labels

5.3 Dark Mode Implementation

Dark mode was implemented using Tailwind CSS's built-in `dark:` utility classes. The application adds a `dark` class to the HTML element whenever dark mode is enabled. User preferences are stored using Local Storage, allowing the selected theme to persist even after the browser is refreshed.

Key features include:

- One-click theme switching.
- Smooth transition animations.
- Persistent user preference.
- Improved readability in low-light environments.

5.4 Responsive Design

The application was designed following a mobile-first approach to ensure compatibility across different devices.

Device	Tailwind Breakpoint	Layout
Mobile	sm	Single-column layout
Tablet	md	Two-column layout
Desktop	lg	Four-column product grid

5.5 User Experience Features

Several user experience improvements were incorporated into the application to enhance usability and accessibility.

- Confirmation dialogs before adding products to the shopping cart.
- Loading indicators during API requests.
- Friendly error messages with retry functionality.
- Smooth hover animations and transitions.
- Visual feedback through alerts and notifications.
- Interactive product cards with scaling and shadow effects.

6. STATE MANAGEMENT

Pinia was used as the centralized state management solution for handling shopping cart data throughout the application. It provides a simple and efficient way to share data between multiple Vue components without requiring complex prop passing.

The cart store manages product items, bundle offers, quantity updates, price calculations, and data persistence.

6.1 Cart Store Structure

```
interface CartItem {  
  
  id: number;  
  
  title: string;  
  
  price: string;  
  
  image: string;  
  
  brand: string;  
  
  quantity: number;  
  
  isBundle?: boolean;  
  
  bundleItems?: any[];  
  
  discount?: number;  
  
}
```

6.2 Store State

The store maintains an array containing all individual products and bundle items added by the user.

```
const cartStore = {  
  
  state: {  
  
    items: CartItem[]  
  
  }  
  
}
```

6.3 Getters

Getters are used to calculate and retrieve derived values from the cart state.

Getter	Purpose
totalItems	Calculates the total number of products in the cart
subtotal	Calculates the total price before applying discounts
bundleDiscount	Calculates the total discount obtained from bundle offers
totalPrice	Calculates the final payable amount after discounts

6.4 Actions

Actions are responsible for modifying cart data and performing operations.

Action	Functionality
addItem(product)	Adds an individual product to the cart
addBundle(bundle)	Adds a discounted product bundle
updateQuantity(id, qty)	Updates the quantity of selected products
removeItem(id)	Removes products from the cart
clearCart()	Clears all cart items after completing an order
saveToLocalStorage()	Stores cart data permanently in the browser
loadCart()	Retrieves previously saved cart data

6.5 Why Pinia Was Selected

Pinia was selected because of the following advantages:

Advantage	Description
Simplicity	Provides an easier API compared with Vuex
TypeScript Support	Offers strong type checking and improved development experience
Vue DevTools Integration	Supports debugging and state inspection
Lightweight	Has a smaller bundle size and better performance
Composition API Support	Works naturally with Vue 3 Composition API

7. CHALLENGES AND SOLUTIONS

During the development process, several technical challenges were encountered. Appropriate solutions were implemented to ensure a smooth user experience and reliable application performance.

Challenge	Solution Implemented
Dark mode text becoming invisible in dropdown menus	Added appropriate Tailwind classes such as <code>bg-white dark:bg-gray-800</code> and <code>text-gray-700 dark:text-gray-300</code> for better visibility.
Product details displaying incorrect information	Corrected route parameter handling and verified product IDs received from the API.
Bundle discounts not applying correctly	Created a separate <code>addBundle</code> action in Pinia to calculate and apply discounts automatically.
Gifts page displaying unrelated grocery products from API	Replaced the page content with a "Coming Soon" message to maintain beauty brand consistency.
Accidental product additions to cart	Added confirmation dialogs before adding products.
Search results displaying unrelated products	Implemented filtering to display only beauty-related products.
Cart data disappearing after refresh	Implemented Local Storage integration with Pinia actions.

8. TECHNOLOGY STACK

The project was developed using modern frontend technologies to create a scalable and efficient Single Page Application.

8.1 Frontend Technologies

Technology	Version	Purpose
Vue 3	3.5.x	Frontend framework using Composition API
TypeScript	5.x	Static type checking and improved code reliability
Tailwind CSS	3.4.x	Responsive and utility-based styling
Vite	5.x	Fast build tool and development server

8.2 State Management and Routing

Technology	Version	Purpose
Pinia	2.x	Global state management for shopping cart
Vue Router	4.x	Client-side routing and navigation

8.3 API and Data Technologies

Technology	Purpose
DummyJSON API	Provides product information through REST API endpoints
Fetch API	Handles HTTP requests between application and API
Local Storage	Stores cart information and user preferences locally

8.4 Development Tools

Tool	Purpose
Visual Studio Code	Primary code editor
Git	Version control system
GitHub	Remote repository management
Chrome DevTools	Debugging and performance testing

9. TESTING AND VALIDATION

Testing was performed manually to verify that all major functionalities worked correctly. Different user scenarios were tested to ensure reliability, responsiveness, and proper user interaction.

9.1 Manual Testing Results

Test Case	Status	Result
API data fetching	✓ Passed	Products are successfully retrieved and displayed
Product navigation	✓ Passed	Selecting a product opens the correct detail page
Add to cart functionality	✓ Passed	Products are added after confirmation
Cart quantity update	✓ Passed	Product quantities update correctly
Remove products from cart	✓ Passed	Products are removed successfully
Bundle discount calculation	✓ Passed	Discounts are correctly applied
Dark mode functionality	✓ Passed	Theme switches and remains saved
Search functionality	✓ Passed	Relevant products are displayed
Checkout process	✓ Passed	Form validation works correctly
Card payment simulation	✓ Passed	Payment process functions correctly
Responsive design	✓ Passed	Application works across different screen sizes

10. CONCLUSION

10.1 Project Summary

The **Oshi Beauty - Luxury Skincare & Makeup E-commerce Single Page Application** successfully demonstrates the development of a modern e-commerce platform using Vue 3, TypeScript, and Tailwind CSS.

The application integrates external API data, provides smooth navigation, manages shopping cart functionality, and delivers a responsive user interface suitable for online beauty shopping.

The following major features were successfully implemented:

- Vue 3 using Composition API and `<script setup>` syntax.
- TypeScript-based type-safe application development.
- Tailwind CSS responsive styling with dark mode support.
- DummyJSON API integration for dynamic product retrieval.
- Vue Router implementation for page navigation.
- Pinia state management for shopping cart operations.
- Bundle offers with automatic discount calculations.
- Real-time product search functionality.
- Checkout and payment simulation.
- Confirmation dialogs for user actions.
- Local Storage-based data persistence.

10.2 Learning Outcomes

Through this project, practical knowledge and experience were gained in the following areas:

- Developing Single Page Applications using Vue 3.
- Implementing reusable components and component-based architecture.
- Writing maintainable and type-safe code using TypeScript.
- Consuming and handling REST APIs.
- Managing application state using Pinia.
- Creating responsive interfaces using Tailwind CSS.
- Implementing routing using Vue Router.
- Using browser storage for persistent data.
- Debugging and solving frontend development challenges.
- Utilizing AI tools to improve development productivity.

11. FUTURE ENHANCEMENTS

Although the current application successfully implements the required functionalities, several additional features can be incorporated in future versions to enhance usability, security, and overall customer experience.

11.1 User Authentication

Implement secure user authentication using JSON Web Tokens (JWT) or Firebase Authentication. This would enable users to create accounts, securely log in, and manage their personal information.

11.2 Order History

Provide registered users with access to their previous orders, allowing them to review purchase history, reorder products, and track deliveries.

11.3 Wishlist Functionality

Allow customers to save their favorite products in a wishlist for future purchases. This feature can improve user engagement and increase customer retention.

11.4 Product Reviews and Ratings

Enable authenticated users to submit ratings and reviews for purchased products, helping future customers make informed purchasing decisions.

11.5 Online Payment Gateway Integration

Replace the simulated payment system with a real payment gateway such as **Stripe** or **PayPal** to facilitate secure online transactions.

11.6 Admin Dashboard

Develop an administrative dashboard to manage products, inventory, customer orders, and promotional offers. This would provide administrators with better control over the e-commerce platform.

11.7 Email Notifications

Implement automated email notifications for order confirmations, shipping updates, promotional offers, and password recovery.

11.8 Product Recommendations

Introduce AI-based recommendation systems that suggest products based on customer browsing history and previous purchases, creating a more personalized shopping experience.

12. REFERENCES

The following resources were used throughout the development of this project.

12.1 API Documentation

- DummyJSON API Documentation. Available at: <https://dummyjson.com/docs>

12.2 Frameworks and Libraries

- Vue.js Official Documentation. Available at: <https://vuejs.org/>
- TypeScript Official Documentation. Available at: <https://www.typescriptlang.org/>
- Tailwind CSS Official Documentation. Available at: <https://tailwindcss.com/>
- Pinia Official Documentation. Available at: <https://pinia.vuejs.org/>
- Vue Router Official Documentation. Available at: <https://router.vuejs.org/>
- Vite Official Documentation. Available at: <https://vitejs.dev/>

12.3 Additional Resources

- Google Fonts (Noto Serif and Plus Jakarta Sans)
- Material Icons
- Unsplash (Reference images and inspiration)

13. ACKNOWLEDGEMENTS

The successful completion of this project would not have been possible without the guidance and support of several individuals and organizations.

Special appreciation is extended to the **DummyJSON** development team for providing a free and reliable REST API that supplied the product data used throughout the application.

I also acknowledges the creators of **Vue.js**, **TypeScript**, **Tailwind CSS**, **Pinia**, **Vue Router**, and **Vite** for developing powerful open-source technologies that made this project possible.

Gratitude is also extended to **Google Fonts**, **Material Icons**, and **Unsplash** for providing typography, icons, and design inspiration that contributed to the visual appearance of the application.

Finally, sincere thanks are extended to family and to my partner for their encouragement and support throughout the development of this project.